

Cheat Sheet for Data Analysis #1

Pandas

Import pandas library

```
import pandas as pd
```

Meaning: Imports the pandas library and gives it the alias “pd” for easier reference

Load a CSV file into a DataFrame

```
df1 = pd.read_csv("Diamond.csv")
```

Meaning: Reads data from a CSV file and stores it in a variable called df1; the file must sit next to the notebook, otherwise the full path is needed

Load a file that uses a semicolon as the separator

```
df1 = pd.read_csv("Diamond.csv", sep=";")
```

Meaning: read_csv expects a comma by default; if the file uses a semicolon, say so with sep=";". Open the file in a text editor and look at line 1 to find out. All the data landing in one column is the sign of a wrong separator

Naming a dataframe

```
df_diamonds = pd.read_csv("Diamond.csv")
```

Meaning: Letters, digits and underscores only; a name cannot start with a digit; no spaces and no hyphens; names are case-sensitive, so df and DF are different. Do not use sum, list, type, max, str, pd, np as names

One file, one name

```
df1 = pd.read_csv("Diamond.csv")  
df2 = pd.read_csv("Wage.csv")
```

Meaning: Loading a second file into the same name replaces the first table silently and without a warning. Two files means two names

Reassign on purpose

```
df_clean = df1.dropna()
```

Meaning: Writing df1 = df1.dropna() is normal practice, but the original table is then gone from memory. A new name keeps both the raw table and the cleaned one

View the shape of the DataFrame (rows, columns)

```
print(df1.shape)
```

Meaning: .shape returns a tuple: (number of rows, number of columns); df1.shape[0] means rows and df1.shape[1] means columns

Get total number of elements (rows × columns)

```
print(df1.size)
```

Meaning: .size returns the total count of all cells in the DataFrame

Display data types of each column

```
print(df1.dtypes)
```

Meaning: `.dtypes` shows the data type (e.g., object, int64, float64) of each column

Show the first 5 rows of the DataFrame

```
print(df1.head(5))
```

Meaning: `.head(n)` displays the first n rows (default is 5)

Show the last 5 rows of the DataFrame

```
print(df1.tail(5))
```

Meaning: `.tail(n)` displays the last n rows (default is 5)

Get concise summary of the DataFrame

```
print(df1.info())
```

Meaning: `.info()` shows column names, non-null counts, data types, and memory usage

Count occurrences of unique values in a column

```
domain_counts = df1['example_column'].value_counts(dropna=True)
```

Meaning: `.value_counts()` returns a Series with frequency counts of unique values; `dropna=True` excludes missing (NaN) values from the count

Additional tip: To count combinations of multiple columns, use: `df1[['column1', 'column2']].value_counts()`

Always check for missing data before analysis

```
print(df1.isnull().sum())
```

Meaning: You can use `df1.isnull().sum()` to see how many missing values per column

Practical Tips

1. **Look at the file before you load it.** Open the CSV in a text editor and read line 1: it tells you the separator and whether there is a header row.
2. **Watch the decimal comma.** If numbers are written as 0,21, the separator has to be a semicolon, otherwise one value becomes two columns. Some regional settings make Excel write files this way.
3. **One column, one type of data.** Do not mix numbers and text in the same column, and leave a cell empty for a missing value rather than writing a word.
4. **Do not edit a CSV in Excel without need.** Excel rewrites dates, drops leading zeros and changes separators. Keep the original file untouched.
5. **Run the same four checks on every new file:** `.shape`, `.dtypes`, `.head()` and `.isnull().sum()`. They take one minute and they catch most broken files.

Remember: the name of a dataframe is only a label, but it is a label you will type fifty times and read again in six months. Spend five seconds choosing it.